DAY 19 – LINUX SECURITY BASICS

Cyber Security Internship – Veda Technology

1. Objective

To understand the fundamentals of Linux security, identify users and groups, examine file permissions, and implement access restrictions using Linux commands. The task also demonstrates how proper permission management helps prevent unauthorized access to files and system resources.

2. Introduction

Linux is an open-source operating system widely used in servers, cloud computing, cybersecurity, and network administration. Security is an important aspect of Linux because multiple users and processes may access the same system.

Linux uses a permission-based access control mechanism to determine who can read, modify, or execute files and directories. Permissions are assigned to three categories: owner, group, and others.

By using commands such as whoami, groups, ls -l, and chmod, users can identify account information, inspect permissions, and control access to important files.

3. Linux File Permission Fundamentals

Linux permissions determine the operations allowed for each user category.

Symbol	Permission	Numeric Value	Description
r	Read	4	Allows reading file contents
w	Write	2	Allows modifying file contents
x	Execute	1	Allows executing a file or accessing a directory
-	No permission	0	Denies the corresponding permission

4. Practical Implementation

Step 1: Identify the Current User

Command:

whoami

Description: Displays the username of the currently logged-in user. This helps identify the account under which commands are executed.

Expected output: The current username, such as user.

Step 2: Identify User Groups

Command:

groups

Description: Displays the groups associated with the current user. Group membership can determine access to shared files and directories.

Expected output: A list of groups to which the current user belongs.

Step 3: Create a Working Directory and File

Commands:

mkdir Day19_LinuxSecurity

cd Day19_LinuxSecurity

touch security.txt

Description:

- mkdir creates a new directory.
- cd changes the current working directory.
- touch creates an empty file if it does not already exist.

Expected result: A directory named Day19_LinuxSecurity containing the file security.txt.

Step 4: View File Permissions

Command:

ls -l

Description: Lists files and directories with detailed information, including permissions, ownership, file size, and modification time.

Step 5: Restrict File Access

Command:

chmod 600 security.txt

Description: Changes the file permissions so that only the owner has read and write access.

Permission breakdown:

User Category	Permission	Numeric Value
Owner	Read and write (rw-)	6
Group	No permission (---)	0
Others	No permission (---)	0

Security benefit: This setting is suitable for files that contain sensitive information and should not be accessible to other users.

Step 6: Allow Read Access to Other Users

Command:

chmod 644 security.txt

Description: Grants read and write permissions to the owner and read-only permissions to the group and others.

User Category	Permission	Numeric Value
Owner	Read and write (rw-)	6
Group	Read only (r--)	4
Others	Read only (r--)	4

Security benefit: This configuration is useful when a file must be readable by other users without allowing them to modify its contents.

Step 7: Create and Execute a Shell Script

Commands:

echo "echo Linux Security Task 19" > demo.sh

chmod 700 demo.sh

./demo.sh

Description:

- echo generates the script content.
- > writes the content into demo.sh.
- chmod 700 grants read, write, and execute permissions only to the owner.
- ./demo.sh executes the script from the current directory.

Expected output:

Linux Security Task 19

Security benefit: Restricting script execution to the owner helps prevent other users from running or modifying the script through ordinary file permissions.

5. Summary of Commands

Command	Purpose	Security Relevance
whoami	Displays the current username	Identifies the active account
groups	Displays group membership	Helps understand group-based access
mkdir	Creates a directory	Organizes files securely
cd	Changes the working directory	Navigates the file system
touch	Creates an empty file	Creates a file for permission testing
ls -l	Displays detailed file information	Inspects file access permissions
chmod 600	Owner-only read and write access	Protects sensitive files
chmod 644	Owner write access; everyone can read	Supports controlled file sharing
chmod 700	Owner-only read, write, and execute access	Restricts script access
./demo.sh	Executes the shell script	Demonstrates controlled script execution

6. Importance of Linux Security

1. **Prevents unauthorized access:** Restricts access to confidential files and directories.
2. **Protects data integrity:** Reduces the risk of unauthorized modifications.
3. **Supports user accountability:** Helps administrators identify users and understand group membership.
4. **Improves system security:** Limits unnecessary permissions and reduces opportunities for misuse.
5. **Protects executable files:** Restricts who can run or modify scripts and programs.
6. **Encourages least privilege:** Gives users only the permissions required for their tasks.

7. Best Practices

- Apply the principle of least privilege when assigning file permissions.
- Use restrictive permissions such as 600 for sensitive files when appropriate.
- Avoid granting write or execute permissions to everyone without a valid reason.

- Review file ownership and permissions regularly.
- Use strong passwords and secure authentication practices.
- Avoid running commands with administrator privileges unless necessary.
- Remember that file permissions are one layer of security; they do not replace encryption, authentication, or system updates.

8. Result and Conclusion

The Linux Security Basics task demonstrated essential Linux commands for user identification, group inspection, file creation, permission analysis, and access control. The ls -l command was used to examine file permissions, while chmod demonstrated how access rights can be restricted or modified.

The exercise showed that permission settings such as 600, 644, and 700 provide different levels of access depending on the file's purpose. It also demonstrated the execution of a shell script with restricted permissions.

In conclusion, understanding Linux permissions and applying appropriate security practices are important skills for protecting files, preventing unauthorized access, and maintaining a secure operating environment.

